

Programming Techniques 2026–2027

Lecture 5: Dynamic Memory and Pointers

Hannu Parviainen

Universidad de la Laguna

October 7, 2026

Dynamic Memory Allocation

Static memory allocation

Concept

- ▶ The array size is fixed at compile time.
- ▶ The memory is reserved when the program or procedure starts.
- ▶ Lifetime = the whole run of the program (or of the procedure).
- ▶ Efficient, but inflexible when the array size is not known in advance.

```
program static_demo
  implicit none
  real, dimension(100) :: x
  integer :: i

  do i = 1, 100
    x(i) = i * 0.1
  end do

  print *, "x(100) = ", x(100)
end program static_demo
```

Dynamic memory allocation

Concept

- ▶ The size and shape are chosen at run time.
- ▶ The memory is allocated with `allocate`.
- ▶ It is released with `deallocate`, or automatically when the array goes out of scope.
- ▶ More flexible, at the cost of a little time for each allocation.

```
program dynamic_demo
  implicit none
  real, dimension(:), allocatable :: y
  integer :: n, i

  print *, "Enter size:"
  read *, n
  allocate(y(n))

  do i = 1, n
    y(i) = i * 0.5
  end do

  print *, "y(n) = ", y(n)
  deallocate(y)
end program dynamic_demo
```

Allocatable Arrays in Fortran

Declaration

```
integer, dimension(:), allocatable :: ages  
real, dimension(:, :), allocatable :: speed
```

Allocation

```
integer :: isize, ierr  
read *, isize  
  
allocate(ages(isize))  
  
allocate(speed(0:isize-1, 10), stat=ierr)  
if (ierr /= 0) error stop 'speed: allocation failed'
```

Note

Without `stat`, a failed allocation stops the program with an error message. With `stat`, the program continues, so you must handle the failure yourself.

Automatic allocation on assignment

- ▶ Since Fortran 2003, assigning to an allocatable array allocates it with the shape of the right-hand side.
- ▶ If the shapes differ, the array is reallocated. So `a = [a, x]` appends an element.
- ▶ Appending this way copies the whole array every time. That is fine for small arrays, but slow inside a long loop.
- ▶ `a(:) = ...` assigns to the existing elements and never reallocates.

```
program auto_alloc
  implicit none
  real, allocatable :: a(:)
  a = [1.0, 2.0, 3.0]      ! allocated with size 3
  a = [a, 4.0]             ! reallocated with size 4
  print *, size(a)        ! 4
  print *, a               ! 1.0  2.0  3.0  4.0
  a = a(2:3)              ! reallocated with size 2
  print *, size(a)        ! 2
  print *, a               ! 2.0  3.0
end program auto_alloc
```

Deallocation and Status

Deallocate

```
if (allocated(ages)) deallocate(ages)
if (allocated(speed)) deallocate(speed)
```

Guidelines

- ▶ A local allocatable array is deallocated automatically when its procedure returns. Allocatable arrays cannot leak memory.
- ▶ Use deallocate to free memory early, for example a large work array.
- ▶ Allocating an array that is already allocated, or deallocating one that is not, is an error. Test with allocated(array).

Allocatable strings

- ▶ `character(len=:)`, allocatable declares a string whose length is set when it is assigned.
- ▶ The string is reallocated whenever it is given a value of a different length.
- ▶ Compare with a fixed-length `character(len=10)`, which is padded with blanks and needs `trim`.

```
program strings
  implicit none
  character(len=:), allocatable :: name
  name = 'Vega'
  print *, name, len(name)
  name = name // ' (alpha Lyr)'
  print *, name, len(name)
end program strings
```

```
$ ./a.out
Vega          4
Vega (alpha Lyr)      16
```

Pointers

Fortran Pointers: Introduction

What is a pointer?

- ▶ A **pointer** is a variable that does not store a value directly, but instead **points to a memory location**.
- ▶ In Fortran, pointers can be associated with
 - ▶ other variables,
 - ▶ array sections,
 - ▶ dynamically allocated memory.

Why are they useful?

- ▶ Allow multiple variables to **share the same data**.
- ▶ Needed for building **linked data structures** (lists, trees, graphs).
- ▶ Provide flexibility similar to references in modern languages.

Python vs. Fortran Variables

Key difference

- ▶ **Python variables** are *references* to objects in memory.
 - ▶ Assigning one variable to another does not copy the object.
 - ▶ Example: `b = a` makes `b` point to the same object as `a`.
- ▶ **Fortran variables** are normally *values* stored directly.
 - ▶ Assigning one variable to another copies the value.
 - ▶ No implicit references like in Python.

Python variables act like Fortran pointers by default, while plain Fortran variables are independent copies.

Pointer Declaration

Definition

- ▶ A variable with the `pointer` attribute.
- ▶ Has static type, kind, and rank determined by its declaration.

Examples

```
real, pointer :: ptor => null()  
real, dimension(:, :), pointer :: ptoa => null()
```

- ▶ `ptor` is a pointer to a scalar real target.
- ▶ `ptoa` is a pointer to a 2D array of reals.
- ▶ A new pointer points nowhere in particular: its status is *undefined*. Initialise it with `=> null()`.

Target Declaration

Targets must have the target attribute

```
real, target :: x, y  
real, dimension(5,3), target :: a, b  
real, dimension(3,5), target :: c
```

- ▶ x or y may become associated with ptor.
- ▶ a, b, or c may become associated with ptoa.

Pointer Manipulation

Operators

- ▶ `=>` pointer assignment: alias pointer with a target.
- ▶ `=` normal assignment: assign value to space pointed at.

Pointer assignment makes the pointer and target reference the same space, while normal assignment changes the value.

Pointer Assignment

```
real, target :: x, y  
real, pointer :: ptor
```

```
x = 3.14159  
ptor => y  
ptor = x
```

- ▶ x and ptor have the same value.
- ▶ ptor is an alias for y, so the assignment sets $y = 3.14159$.
- ▶ Changing x later does not change ptor or y.

Dynamic Targets

Targets can be created dynamically by allocation

```
allocate(ptor, stat=ierr)
allocate(ptoa(n*n, 2*k-1), stat=ierr)
```

- ▶ First: allocates a scalar real as target of ptor.
- ▶ Second: allocates a rank-2 real array as target of ptoa.
- ▶ Allocating a pointer that is already associated is not an error. But if nothing else points to its old target, that memory is lost: a **memory leak**.

Association Status

Test with `associated()`

```
associated(ptoa)      ! is it associated with any target?
associated(ptoa, arr) ! is it associated with arr?
```

- ▶ Returns `.true.` if the pointer is associated.
- ▶ The second form tests association with a specific target.
- ▶ The test is only valid when the status of the pointer is defined. This is another reason to initialise pointers with `null()`.

Pointer Disassociation

Nullification

```
nullify(ptor)      ! or: ptor => null()
```

- ▶ Breaks the pointer–target connection. The target itself is not changed.

Deallocation

```
deallocate(ptoa, stat=ierr)
```

- ▶ Breaks the connection and frees the target.
- ▶ Only for targets that were created with `allocate`. Deallocating a pointer to an ordinary variable is an error.
- ▶ Any other pointer to the same target is left **dangling**: it points to memory that no longer exists.

Allocatable or pointer?

Use allocatable arrays by default

They are freed automatically, so they cannot leak or dangle. The compiler also knows that they do not overlap other variables, which lets it produce faster code.

Use pointers when you need what only they can do

Giving a second name to existing data, such as a section of a large array. Swapping large arrays without copying them. Building linked structures such as lists and trees.

Pointer pitfalls

- ▶ **Memory leak:** the last pointer to allocated memory is redirected or nullified before the memory is deallocated.
- ▶ **Dangling pointer:** the target is deallocated, or goes out of scope, while a pointer still points to it.

Practical Example: Swapping Arrays without Copying

- ▶ Heat diffusion on a grid: each new value is the mean of its four neighbours.
- ▶ Every iteration needs the previous grid and the next one.
- ▶ Swapping the two pointers swaps their roles, and no data are copied.
- ▶ The loop ends when the largest change, `maxval(abs(...))`, is small enough. Here that takes 1898 iterations.

```
program jacobi
  implicit none
  integer, parameter :: n = 100
  real, target :: grid1(n,n), grid2(n,n)
  real, pointer :: old(:,,:), new(:,,:), tmp(:,,:)
  integer :: iter
  grid1 = 0.0
  grid1(1,:) = 1.0           ! hot top edge
  grid2 = grid1
  old => grid1
  new => grid2
  do iter = 1, 100000
    new(2:n-1,2:n-1) = (old(1:n-2,2:n-1) + old(3:n,2:n-1) &
                        + old(2:n-1,1:n-2) + old(2:n-1,3:n)) / 4
    if (maxval(abs(new - old)) < 1.0e-4) exit
    tmp => old                ! swap the roles, copy nothing
    old => new
    new => tmp
  end do
  print *, 'Converged after', iter, 'iterations'
end program jacobi
```

Exercises: a growing array

Tasks

1. Write a program that reads real numbers from standard input until the input ends, and stores them in an allocatable array. Start from an empty array, `allocate(a(0))`, and append each new value with `a = [a, x]`.
2. Print the number of values, their mean, and the values in reverse order.
3. Test the program by redirecting a file to its standard input.
4. What does the program do if the input is empty? Make it stop with a clear message.
5. Appending copies the whole array each time. How could you read a very long file more efficiently?

Exercises: pointers

Tasks

1. Declare two real target arrays, `a` and `b`, with different values, and two pointers, `p` and `q`, initialised with `null()`.
2. Point `p` at `a` and `q` at `b`. Change one element through `p` and print `a`. What happened?
3. Swap the two pointers, so that `p` points at `b` and `q` at `a`, without copying any array.
4. Confirm the result with `associated(p, b)` and `associated(q, a)`.
5. Allocate a new target for `p` with `allocate`. Which of the pitfalls from the lecture can now happen, and how do you avoid them?

Exercises: sigma clipping

Tasks

1. Write a module with a function `clip(x, nsigma)` that returns the elements of the real array `x` that lie within `nsigma` standard deviations of its mean.
2. The size of the result is only known at run time, so make the function result allocatable. Hint: count the elements to keep, allocate the result, and then copy them.
3. Test it with `[10.1, 9.8, 10.3, 9.9, 55.0, 10.0, 10.2]` and `nsigma = 2.0`. Which value is removed?
4. Store the result in an allocatable array in the main program. Do you need to allocate that array yourself?
5. Repeat the clipping until no more values are removed, and print the mean of what is left.

Exercises: a histogram

Tasks

1. Write a program that reads integers from standard input until the input ends, and stores them in an allocatable array.
2. Allocate a second array, `counts`, whose lower and upper bounds are the smallest and the largest value that was read.
3. Count how many times each value occurs, so that `counts(v)` is the number of times the value `v` was read. Print every value with its count.
4. Print the bounds and the size of `counts` with `lbound`, `ubound`, and `size`.
5. What happens with empty input? Make the program stop with a clear message.

Exercises: a window into an image

Tasks

1. Read two integers, `nx` and `ny`, allocate a real array `image(nx,ny)` with the target attribute, and set it to zero.
2. Point a two-dimensional pointer `window` at the central part of the image, `image(nx/4+1:3*nx/4, ny/4+1:3*ny/4)`. Print its shape and its lower bounds.
3. Set `window = 1.0` and print the sum of the whole image. What does the result tell you?
4. Point a one-dimensional pointer at every second element of the first column of the image, set those elements to 5.0, and print the first column.
5. Deallocate the image. What does `associated(window)` return now, and can you trust it? What should you do with the pointers?